

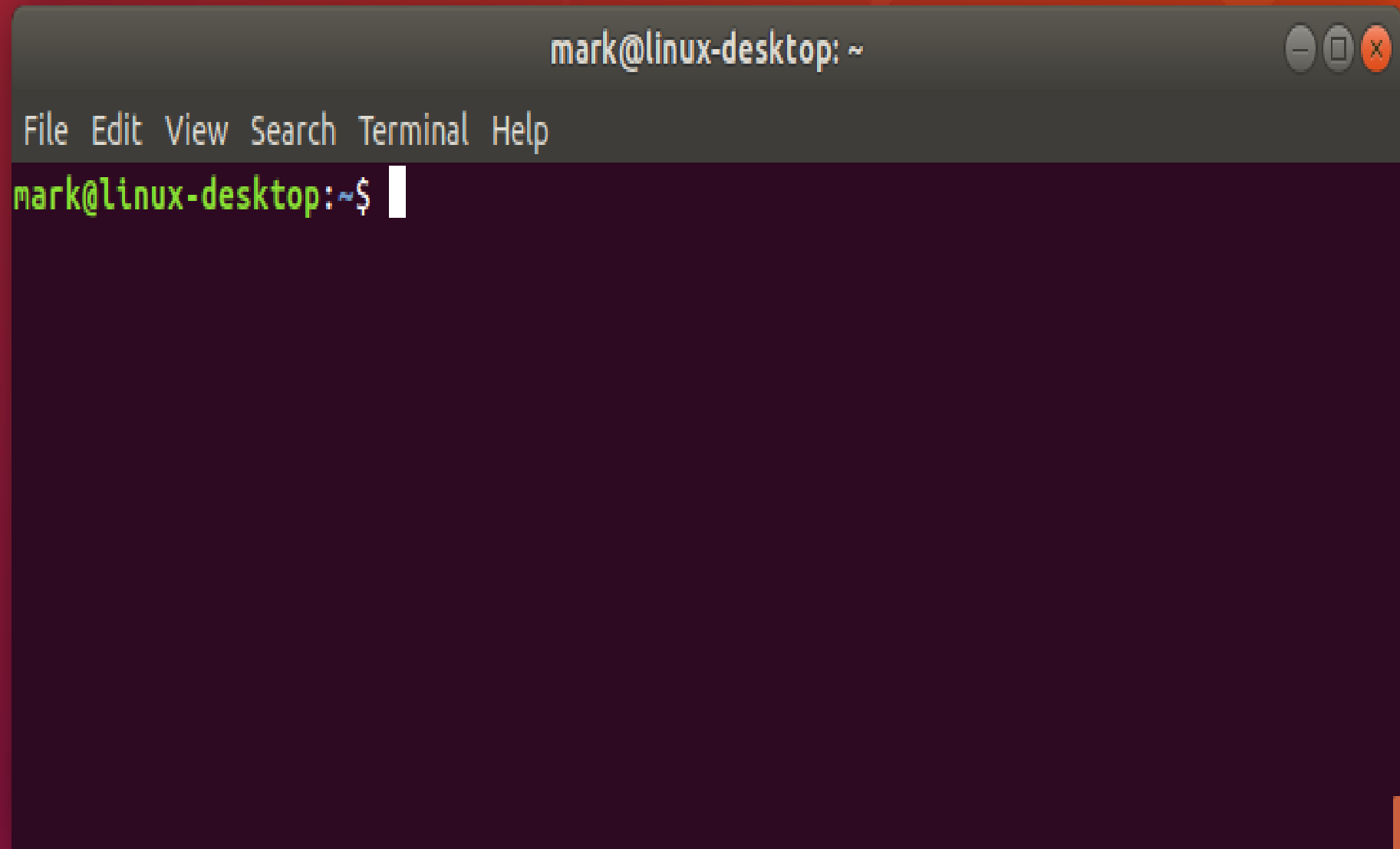
Chapter 6: Command Line Operations

Learning Objectives

By the end of this chapter, you should be able to:

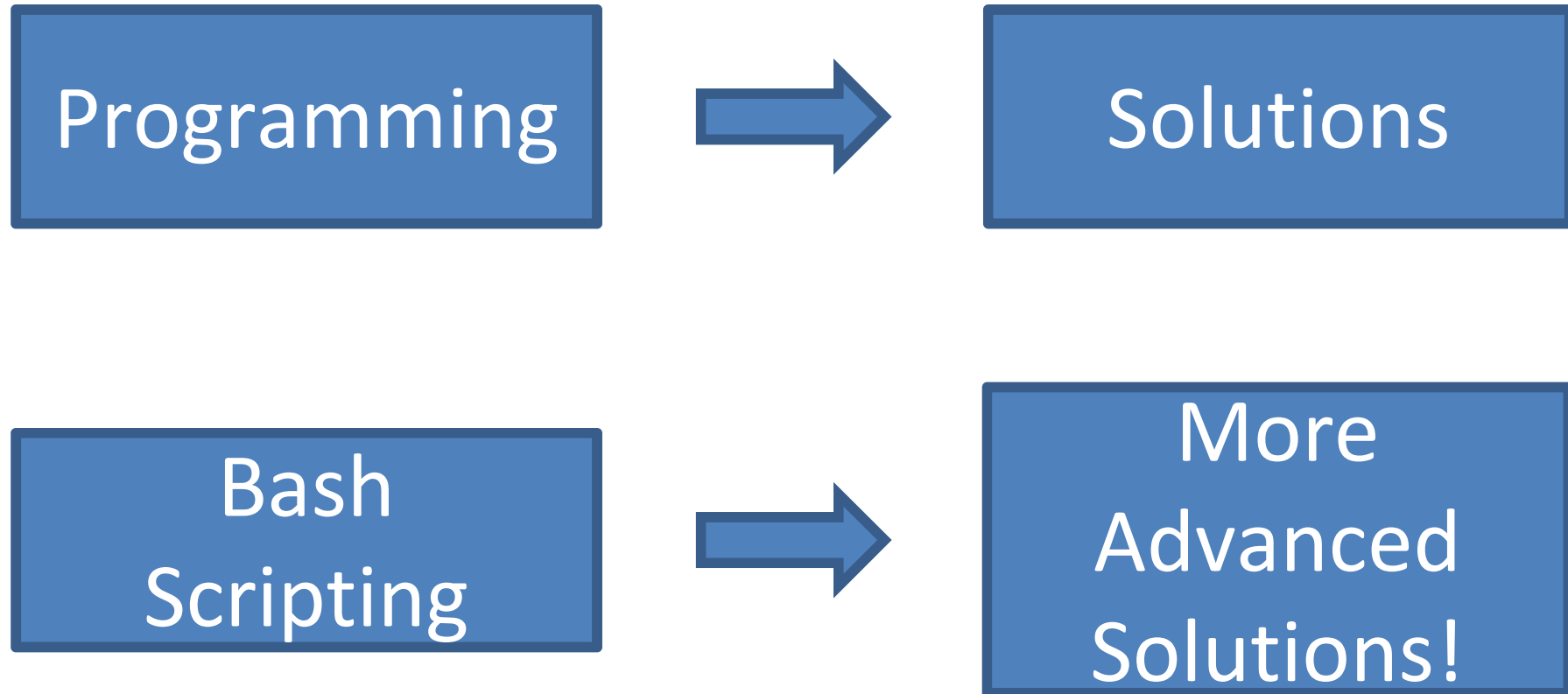
- Use the command line to perform operations in Linux.
- Search for files.
- Create and manage files.
- Install and update software.

Learning Objectives

A screenshot of a Linux terminal window. The window title bar shows 'mark@linux-desktop: ~' and standard window controls. The menu bar includes 'File', 'Edit', 'View', 'Search', 'Terminal', and 'Help'. The terminal text shows the prompt 'mark@linux-desktop:~\$' with a white cursor. The background of the terminal is dark purple.

```
mark@linux-desktop: ~
File Edit View Search Terminal Help
mark@linux-desktop:~$
```

Introduction to the Command Line



WHY ?!!!?

Introduction to the Command Line

Send commands directly to the Operating System !

Access and configure all resources on your PC (devices – services – files – data ...)

Linux shell (bash) have better features than Windows shell (Power shell)

Introduction to the Command Line

“graphical user interfaces make easy tasks easier, while command line interfaces make difficult tasks possible”

The command line interface provides the following advantages:

- No GUI overhead is incurred.
- Virtually any and every task can be accomplished while sitting at the command line.
- You can implement scripts for often-used (or easy-to-forget) tasks and series of procedures.
- You can sign into remote machines anywhere on the Internet.
- You can initiate graphical applications directly from the command line instead of hunting through menus.
- While graphical tools may vary among Linux distributions, the command line interface does not.

Some Basic Utilities

There are some basic command line utilities that are used constantly like:

- **cat**: used to type out a file (or combine files)
- **head**: used to show the first few lines of a file
- **tail**: used to show the last few lines of a file
- **man**: used to view documentation.

The Command Line

Most input lines entered at the shell prompt have three basic elements:

- Command
- Options
- Arguments.

The command is the name of the program you are executing. It may be followed by one or more options (or switches) that modify what the command may do. Options usually start with one or two dashes, for example, **-p** or **-print**, in order to differentiate them from arguments, which represent what the command operates on.

sudo

- Provides the user with administrative (admin) privileges.
- On your own systems, you may need to set up and enable sudo to work correctly.

Example:

- Try running the following command: `ls -ls /root`
- Now try: `sudo ls -ls /root`

Steps for Setting Up and Running sudo

If your system does not already have sudo set up and enabled, you need to do the following steps:

- 1** You will need to make modifications as the administrative or superuser, root. At the command line prompt, type **su** and press **Enter**. You will then be prompted for the root password, so enter it and press **Enter**.
- 2** Now, you need to create a configuration file to enable your user account to use sudo. Typically, this file is created in the **/etc/sudoers.d/** directory with the name of the file the same as your username. If your username is "student", create the configuration file using:

```
echo "student ALL=(ALL) ALL" > /etc/sudoers.d/student
```

- 3** Finally, some Linux distributions will complain if you do not also change permissions on the file by doing:

```
chmod 440 /etc/sudoers.d/student
```

Locating Applications

One way to locate programs is to employ the **which** utility. For example, to find out exactly where the diff program resides on the filesystem:

```
$ which diff  
/usr/bin/diff
```

If **which** does not find the program, **whereis** is a good alternative because it looks for packages in a broader range of system directories:

```
$ whereis diff  
diff: /usr/bin/diff /usr/share/man/man1/diff.1.gz
```

Accessing Directories

- When you first log into a system or open a terminal, the default directory should be your Home directory.
- You can print the exact path of this by typing **echo \$HOME**.

The following commands are useful for directory navigation:

Command	Result
pwd	Displays the present working directory
cd ~ or cd	Change to your home directory (shortcut name is ~(tilde))
cd ..	Change to parent directory (..)
cd -	Change to previous directory (- (minus))

Understanding Absolute and Relative Paths

There are two ways to identify paths:

- **Absolute pathname:** An absolute pathname begins with the root directory and follows the tree, branch by branch, until it reaches the desired directory or file. Absolute paths always start with `/`.

```
$ cd /usr/bin
```

- **Relative pathname:** A relative pathname starts from the present working directory. Relative paths never start with `/`.

```
$ cd ../../usr/bin
```

Exploring the Filesystem

The **tree** command is a good way to get a bird's-eye view of the filesystem tree. Use **tree -d** to view just the directories and to suppress listing file names.

The following commands can help in exploring the filesystem:

cmd	Usage
cd /	Changes your current directory to the root (/) directory (or path you supply)
ls	List the contents of the present working directory
ls -a	List all files, including hidden files and directories (those whose name start with .)
tree	Displays a tree view of the filesystem

touch

touch is often used to set or update the access, change, and modify times of files. By default, it resets a file's timestamp to match the current time.

However, you can also create an empty file using **touch**:

```
$ touch <filename>
```

This is normally done to create an empty file as a placeholder for a later purpose.

touch provides several useful options. For example, the **-t** option allows you to set the date and timestamp of the file to a specific value, as in:

```
$ touch -t 12091600 myfile
```

This sets the **myfile** file's timestamp to 4 p.m., December 9th (12 09 1600).

mkdir and rmdir

mkdir is used to create a directory:

- **mkdir sampdir**: It creates a sample directory named **sampdir** under the current directory.
- **mkdir /usr/sampdir**: It creates a sample directory called **sampdir** under **/usr**.

Removing a directory is done with **rmdir**. The directory must be empty or the command will fail. To remove a directory and all of its contents you have to do **rm -rf**.

Moving, Renaming or Removing a File

Note that **mv** does double duty, in that it can:

- Simply rename a file
- Move a file to another location, while possibly changing its name at the same time.

If you are not certain about removing files that match a pattern you supply, it is always good to run **rm** interactively (**rm -i**) to prompt before every removal.

Command	Usage
mv	Rename a file
rm	Remove a file
rm -f	Forcefully remove a file
rm -i	Interactively remove a file

Renaming or Removing a Directory

rmdir works only on empty directories; otherwise you get an error.

While typing **rm -rf** is a fast and easy way to remove a whole filesystem tree recursively, it is extremely dangerous and should be used with the utmost care, especially when used by root (recall that recursive means drilling down through all sub-directories, all the way down a tree).

Command	Usage
mv	Rename a directory
rmdir	Remove an empty directory
rm -rf	Forcefully remove a directory recursively

Modifying the Command Line Prompt

The **PS1** variable is the character string that is displayed as the prompt on the command line. Most distributions set **PS1** to a known default value, which is suitable in most cases. However, users may want custom information to show on the command line. To check the content of **PS1** then modify it:

```
$ echo $PS1
```

```
$ PS1="\u@\h \ $ "
```

Standard File Streams

- In Linux, all open files are represented internally by what are called file descriptors (represented by numbers).
- When commands are executed, by default there are three standard file streams (or descriptors) always open for use:

Name	Symbolic Name	Value	Example
standard input	stdin	0	keyboard
standard output	stdout	1	terminal
standard error	stderr	2	log file

I/O Redirection

If we have a program called **do_something** that reads from stdin and writes to stdout and stderr, we can change its input source by using the less-than sign (<) followed by the name of the file to be consumed for input data:

```
$ do_something < input-file
```

If you want to send the output to a file, use the greater-than sign (>) as in:

```
$ do_something > output-file
```

If you want to redirect stderr to a separate file:

```
$ do_something 2> error-file
```

send anything written to file descriptor 2 (stderr) to the same place as file descriptor 1 (stdout):

```
$ do_something >& all-output-file
```

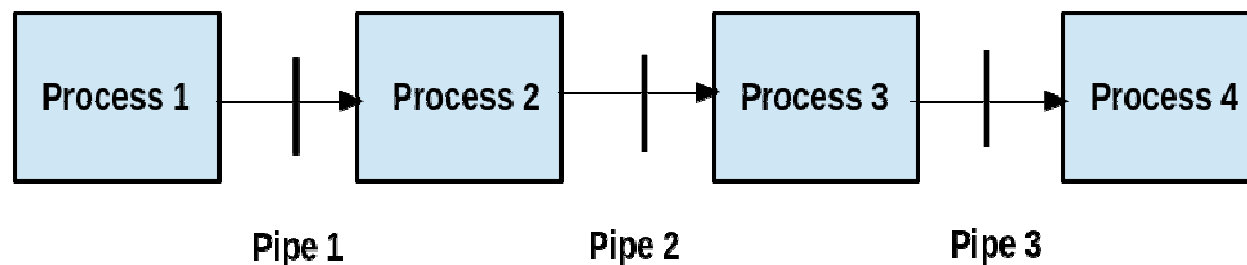
Pipes

The UNIX/Linux philosophy is to have many simple and short programs (or commands) cooperate together to produce quite complex results.

You can pipe the output of one command or program into another as its input using the vertical symbol `|` between commands:

```
$ command1 | command2 | command3
```

This is called a pipeline, and allows Linux to combine the actions of several commands into one.



Searching for files: locate

- The **locate** utility program performs a search taking advantage of a previously constructed database of files and directories on your system, matching all entries that contain a specified character string. This can sometimes result in a very long list.
- To get a shorter (and possibly more relevant) list, we can use the **grep** program as a filter. **grep** will print only the lines that contain one or more specified strings, as in:

```
$ locate zip | grep bin
```

which will list all the files and directories with both **zip** and **bin** in their name.

Wildcards and Matching File Names

You can search for a filename containing specific characters using wildcards.

Wildcard	Result
?	Matches any single character
*	Matches any string of characters
[set]	Matches any character in the set of characters, for example [adf] will match any occurrence of "a", "d", or "f"
[!set]	Matches any character not in the set of characters

Using find

- When no arguments are given, **find** lists all files in the current directory and all of its subdirectories.
- Commonly used options:
 - **-name**: only list files with a certain pattern in their name
 - **-iname**: also ignore the case of file names
 - **-type**: which will restrict the results to files of a certain specified type, such as **d** for directory, **l** for symbolic link, or **f** for a regular file, etc.

Using find

Searching for files and directories named gcc:

```
$ find /usr -name gcc
```

Searching only for directories named gcc:

```
$ find /usr -type d -name gcc
```

Searching only for regular files named gcc:

```
$ find /usr -type f -name gcc
```

Using Advanced find Options

Another good use of find is being able to run commands on the files that match your search criteria. The **-exec** option is used for this purpose.

To find and remove all files that end with .swp:

```
$ find -name "*.swp" -exec rm {} ';' 
```

- **{}**: will be filled with all the file names that result from the find expression, and the preceding command will be run on each one individually.
- Please note that you have to end the command with either **;** (including the single-quotes) or **\;**
- You can use the **-ok** option, which behaves the same as **-exec**, except that find will prompt you for permission before executing the command.

Using Advanced find Options

```
$ find -name "*.swp" -exec rm {} ';' |
```



Finds and removes files
that ends with **.swp**



***.swp**

Finding and Removing Files that Ends with .swp

Finding Files Based on Time

```
$ find / -ctime 3
```

- **-ctime** is often, but not necessarily, when the file was first created.
- You can also search for accessed/last read (**-atime**) or modified/last written (**-mtime**) times.
- **n** is the number of days:
 - **n** means exactly that value
 - **+n** means greater than that number
 - **-n** means less than that number
- There are similar options for times in minutes (as in **-cmin**, **-amin**, and **-mmin**).
- To search files that are modified today:

```
$ find -type f -mtime 0
```

Finding Files Based on Size

```
$ find / -size 0
```

- the size here is in 512-byte blocks, by default.
- you can also specify bytes (c), kilobytes (k), megabytes (M), gigabytes (G), etc.
- file sizes can also be exact numbers **n**, **+n** or **-n**.

For example, to find files greater than 10 MB in size and running a command on those files:

```
$ find / -size +10M -exec command {} ';' 
```

- use `man find` to get more help.

Package Management Systems on Linux

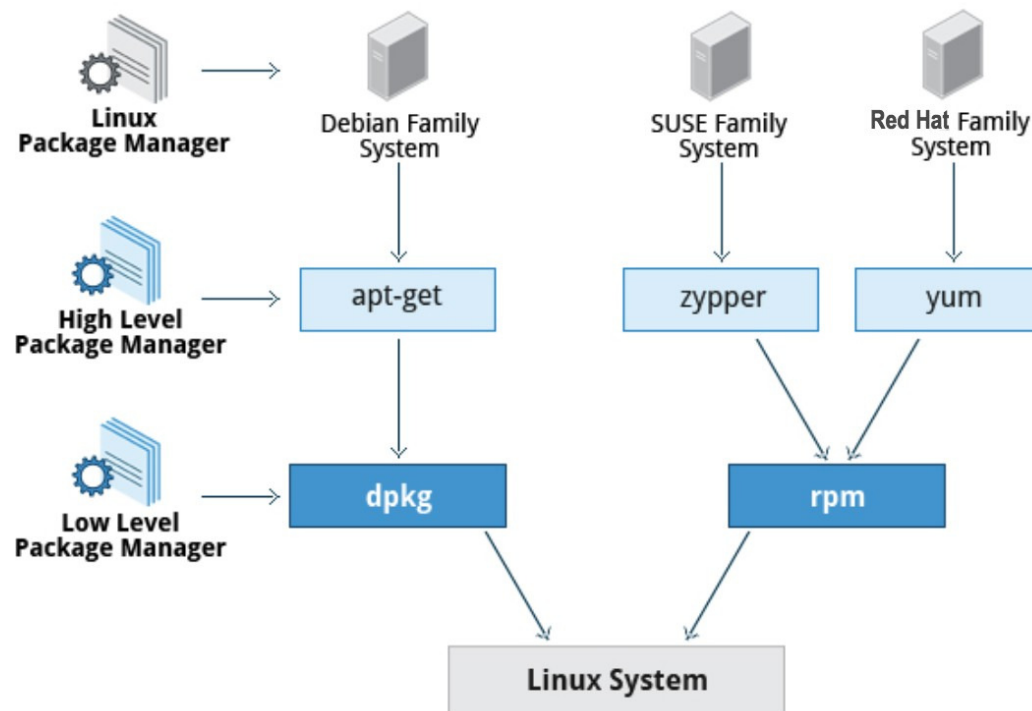
There are two broad families of package managers:

- those based on Debian
- those which use RPM as their low-level package manager
- The two systems are incompatible, but broadly speaking, provide the same features and satisfy the same needs.
- you will learn how to install, remove, or search for packages from the command line using these two package management systems.

Package Managers: Two Levels

Both package management systems operate on two distinct levels:

- a low-level tool (such as dpkg or rpm)
- a high-level tool (such as apt-get, yum, dnf or zypper)
- Most of the time users need to work only with the high-level tool, which will take care of calling the low-level tool as needed.



Working With Debian Package Management Systems

Operation	deb
Install package	<code>dpkg --install foo.deb</code>
Install package, dependencies	<code>apt-get install foo</code>
Remove package	<code>dpkg --remove foo.deb</code>
Remove package, dependencies	<code>apt-get autoremove foo</code>
Update package	<code>dpkg --install foo.deb</code>
Update package, dependencies	<code>apt-get install foo</code>
Update entire system	<code>apt-get dist-upgrade</code>
Show all installed packages	<code>dpkg --get-selections</code>
Get information on package	<code>dpkg --get-architecture foo</code>
Show packages named foo	<code>dpkg --get-architecture foo</code>
Show all available packages	<code>dpkg --get-architecture foo</code>
What package is file part of?	<code>dpkg --get-architecture file</code>

Low-Level Debian Package Management with dpkg

- To get a list of all the packages on the system:

```
$ dpkg --list
```

- To get a more compact view

```
$ dpkg --list | less
```

- To search for a specific package named bzip2 for example

```
$ dpkg --list | grep bzip2
```

- To see what's contained in that package:

```
$ dpkg --getfiles bzip2 | less
```

High-Level Package Management with apt on Ubuntu

- To search for all the packages that contain wget2

```
$ sudo apt-cache search wget2
```

- To install the package:

```
$ sudo apt-get install wget2-dev
```

- To remove the package:

```
$ sudo apt-get remove wget2
```